



VORTEX

Actor Model Support on Vortex

Students: <Hanxiang Hao>
Mentors: <Akihiro Hayashi, Hyesoon Kim>



Georgia Tech
Open Source Program Office

Project Overview

GPUs are built for regular, lockstep work. Irregular workloads, such as graph analytics, sparse data, anything with unpredictable communication, map badly onto them, and most of the machine goes to waste.

The **actor model** is a different way to program: each actor owns its private data and talks to others only by messages, never by sharing memory. That fits irregular communication naturally, and it already scales these workloads well on CPU clusters. But no GPU has hardware support for it.

Vortex is an open-source RISC-V GPGPU. Because its ISA and microarchitecture are open, we can do what's impossible on a commercial GPU: change the hardware itself.

Goal: identify the runtime and ISA changes Vortex needs to host actors efficiently, and build the smallest prototype that proves it.

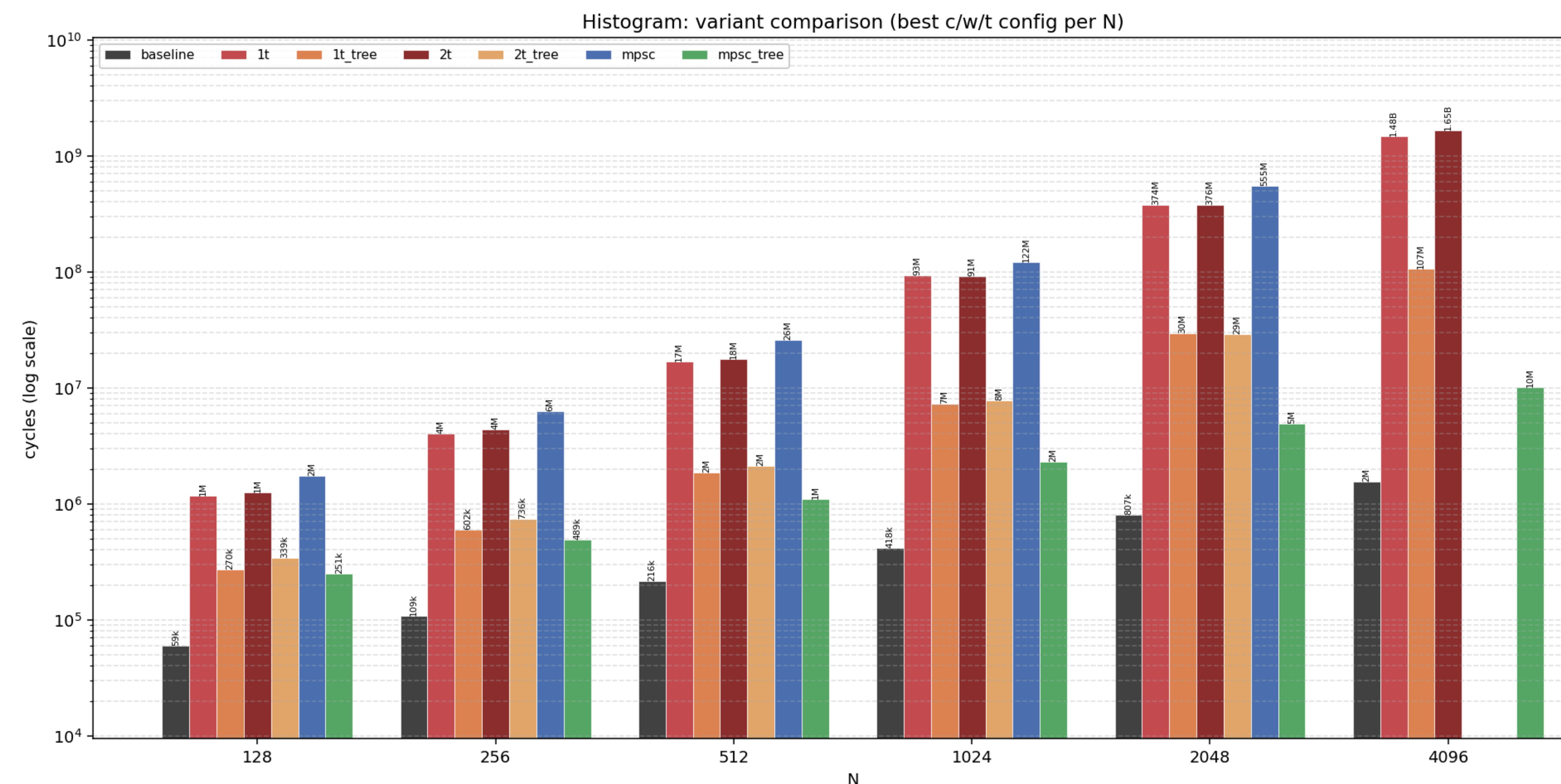
Goals and Milestones

Phase	Focus	Key Tasks & Details
Phase 1	Runtime Port	pthread → vx_spawn; locked queues → lock-free SPSC ring buffers
Phase 2	Characterization	7-phase cycle profiling; 2 irregular workloads, 1024+ actors
Phase 3	Software Opt.	Tree barrier, MPSC buffers; multi-threaded actors, interleaved scheduling
Phase 4	Hardware Accel.	Scratchpad mailboxes, per-actor inbox; msg_wait doorbell
Phase 5	Conclusion	

Open Source Outcomes

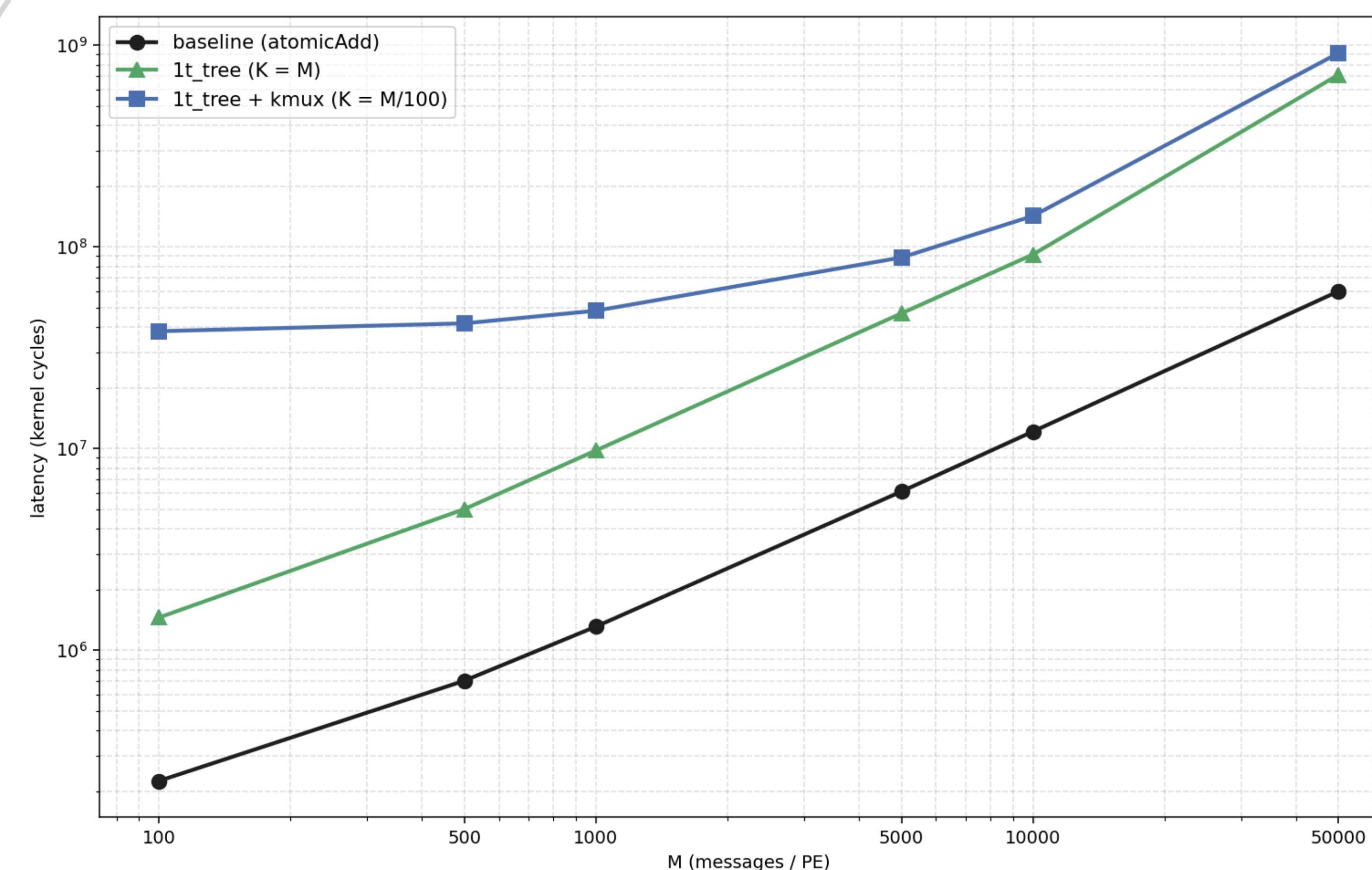
- **vortex-actor** — open fork of Vortex with the actor runtime and 6 regression tests (histogram, index-gather, and microbenchmarks)
- Built on **HCLib-Actor** (hclib-actor.cc.gatech.edu) and Vortex (vortexgpgpu.com/vortex)
- Reusable **profiling infrastructure**: a 7-phase cycle-counter methodology for attributing actor runtime cost, applicable to future actor-on-GPU work

Highlights and Accomplishments – Software Runtime & Characterization



- Ported the HCLib-Actor selector runtime to Vortex: **pthread** → vx_spawn, **SafeQueue** → **lock-free SPSC ring buffers** in global memory
- Built 7-phase cycle profiling; measured histogram and index-gather up to 1024+ actors
- **Key finding: cost is dominated by the communication layer, not computation**
- histogram → send_done (~61%): the all-to-all "done" broadcast, $O(N^2)$ messages
- index-gather → empty_poll (~57%): actors scan all N queues with no arrival signal
- Optimization variants: **tree-based barrier** ($O(N^2) \rightarrow O(N)$), **MPSC buffers** ($O(N)$ scan → $O(1)$, at the cost of atomics), **2-thread actors**, **interleaved (kmux) scheduling**
- Result: the tree barrier is a large win, but the best variant remains **~5x the atomicAdd baseline**

Highlights and Accomplishments – Hardware Acceleration



- **Code-level analysis** of the SPSC mailbox runtime — mapped each profiling phase to its root cause in the poll loop: every iteration scans all N sender queues, with no arrival notification
- **Diagnosis — the bottleneck is memory-bound**: mailbox buffers exceed L1/L2 and live in global memory; a single thread already saturates memory bandwidth, so 2-thread actors and interleaved scheduling have nothing left to overlap. **Software scheduling cannot cross this wall.**

Proposed hardware direction:

- **Scratchpad mailboxes** — move message buffers from global memory onto Vortex's per-core scratchpad, attacking the memory wall directly
- **Per-actor hardware inbox** — collapse the $O(N)$ queue scan to a single $O(1)$ check
- **msg_wait doorbell** — sleep until a message arrives, eliminating idle polling

Target workload: index-gather — its request/response pattern is something atomicAdd cannot express, so it is where actors and hardware support can actually win

Future Work

Software

- Local message aggregation: $M \rightarrow N$ messages per actor
- True compute/communication overlap — needs multi-threaded actors
- Complete index-gather sweep; scale beyond 4096 actors

Hardware

- Prototype scratchpad mailboxes + msg_wait doorbell in SimX
- Evaluate against the optimized software baseline
- Assess ISA extension cost

Workload

Target patterns atomicAdd can't express — index-gather, integer sort

